

LAPORAN PRAKTIKUM TUGAS IoT II



Oleh:

NAMA : AKBAR BETHARA IMANI

NPM : 25782088

KELAS : TRI – 3D

Dosen:

Dr. Ir. Septafiansyah Dwi Saputra, S.T.

Agus Ambarwari, S.Pd., M.Kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026

BAB 1. RANGKAIAN

Sebuah LED ditambahkan pada NodeMCU. Kaki anoda (positif, kaki panjang) LED dihubungkan ke pin D1 (GPIO 5). Kaki katoda (negatif, kaki pendek) LED dihubungkan ke salah satu kaki resistor 220 Ohm sebagai pengaman arus, lalu kaki resistor lainnya dihubungkan ke GND.

BAB 2. PROGRAM

Program terdiri dari sisi server (C++ pada NodeMCU) dan sisi antarmuka web (HTML dan JavaScript) yang disimpan di dalam variabel `index_html` pada program yang sama.

```
#include <ESP8266WiFi.h>
#include <ESPAsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <DHT.h>

const char* ssid = "Redmi 12";
const char* password = "22345678";

// Konfigurasi Pin
const byte dhtPin = 2;           // D4 (GPIO 2)
const byte buttonPin = 4;        // D2 (GPIO 4)
const byte ledPin = 12;          // D6 (GPIO 12) -> LED ON/OFF
const byte pwmPin = 5;           // D1 (GPIO 5) -> LED dimmer (PWM)

DHT dht(dhtPin, DHT22);

// Variabel Pelacak Status (State & Cache)
bool ledState = false;
int pwmValue = 0;
String currentTemp = "--";
int buttonState;
int lastButtonState = LOW;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;
unsigned long lastTime = 0;

// Inisialisasi Async Web Server (port 80) & WebSocket (rute /ws)
AsyncWebServer server(80);
AsyncWebSocket ws("/ws");
```

```
// ----- HTML & JAVASCRIPT (FRONT-END) -----
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-
scale=1">
  <title>Real-Time IoT Web</title>
  <style>
    body { font-family: Arial; text-align: center; }
    .card { background: #f0f0f0; margin: 20px auto; padding: 20px;
max-width: 300px; border-radius: 10px; }
    button { padding: 15px 30px; font-size: 20px; border-radius:
5px; cursor: pointer; color: white;}
    .btn-on { background-color: #4CAF50; }
    .btn-off { background-color: #f44336; }
    input[type=range] { width: 100%; }
  </style>
</head>
<body>
  <h1>Smart Room</h1>
  <div class="card">
    <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
  </div>
  <div class="card">
    <h2>LED: <span id="ledStatus">OFF</span></h2>
    <button id="toggleBtn" class="btn-off"
onclick="toggleLed()">Turn ON</button>
  </div>
  <div class="card">
    <h2>Dimmer LED</h2>
    <input type="range" min="0" max="1023" value="0" id="pwmSlider"
oninput="sendPWM(this.value)">
    <h2>PWM: <span id="pwmValue">0 (0%)</span></h2>
  </div>

  <script>
    var gateway = `ws://${window.location.hostname}/ws`;
    var websocket;

    window.addEventListener('load', onLoad);
    function onLoad(event) { initWebSocket(); }

    function initWebSocket() {
      websocket = new WebSocket(gateway);
```

```

        websocket.onopen      = onOpen;
        websocket.onclose     = onClose;
        websocket.onmessage   = onMessage;
    }

    function onOpen(event) { console.log('WebSocket Terkoneksi'); }
    function onClose(event) { setTimeout(initWebSocket, 2000); }

    function toggleLed(){
        websocket.send('toggle');
    }

    // Dipanggil saat slider digeser: kirim paket berformat
    "pwm,nilai"
    function sendPWM(value) {
        showPWM(value);
        if (websocket.readyState == WebSocket.OPEN) {
            websocket.send('pwm,' + value);
        }
    }

    function showPWM(value) {
        var persen = Math.round(value / 1023 * 100);
        document.getElementById('pwmValue').innerHTML = value + ' (' +
persen + '%)';
    }

    function onMessage(event) {
        var dataObj = JSON.parse(event.data);

        if(dataObj.suhu !== undefined) {
            document.getElementById('tempValue').innerHTML =
dataObj.suhu;
        }

        if(dataObj.pwm !== undefined) {
            document.getElementById('pwmSlider').value = dataObj.pwm;
            showPWM(dataObj.pwm);
        }

        if(dataObj.led !== undefined) {
            var btn = document.getElementById('toggleBtn');
            var status = document.getElementById('ledStatus');
            if(dataObj.led == "1"){
                status.innerHTML = "ON";
            }
        }
    }

```

```

        btn.innerHTML = "Turn OFF";
        btn.className = "btn-on";
    } else {
        status.innerHTML = "OFF";
        btn.innerHTML = "Turn ON";
        btn.className = "btn-off";
    }
    }
}
</script>
</body>
</html>
)rawliteral";

// ----- BACK-END & WEBSOCKET LOGIC -----

void notifyClients() {
    String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) +
"\", ";
    jsonString += "\"suhu\":\"" + currentTemp + "\", ";
    jsonString += "\"pwm\":\"" + String(pwmValue) + "\"}";
    ws.textAll(jsonString);
}

void handleWebSocketMessage(void *arg, uint8_t *data, size_t len) {
    AwsFrameInfo *info = (AwsFrameInfo*)arg;
    if (info->final && info->index == 0 && info->len == len && info-
>opcode == WS_TEXT) {
        data[len] = 0;
        String pesan = (char*)data;

        if (pesan == "toggle") {
            ledState = !ledState;
            notifyClients();
        }
        // Paket slider berformat "pwm,512"
        else if (pesan.startsWith("pwm,")) {
            int nilaiPWM = pesan.substring(4).toInt(); // ambil angka
setelah koma
            nilaiPWM = constrain(nilaiPWM, 0, 1023);
            pwmValue = nilaiPWM;
            analogWrite(pwmPin, pwmValue); // atur kecerahan
LED
            Serial.printf("PWM diterima: %d\n", pwmValue);
        }
    }
}

```

```

    }
}

void onEvent(AsyncWebSocket *server, AsyncWebSocketClient *client,
    AwsEventType type,
    void *arg, uint8_t *data, size_t len) {
    switch (type) {
        case WS_EVT_CONNECT:
            Serial.printf("Client WebSocket #%u terhubung\n", client-
>id());
            notifyClients();
            break;
        case WS_EVT_DISCONNECT:
            Serial.printf("Client WebSocket #%u terputus\n", client-
>id());
            break;
        case WS_EVT_DATA:
            handleWebSocketMessage(arg, data, len);
            break;
        default:
            break;
    }
}

void setup() {
    Serial.begin(115200);

    pinMode(buttonPin, INPUT);

    pinMode(ledPin, OUTPUT);
    digitalWrite(ledPin, LOW);

    pinMode(pwmPin, OUTPUT);
    analogWriteRange(1023);    // resolusi PWM 0-1023 sesuai slider
    analogWrite(pwmPin, 0);

    dht.begin();

    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) { delay(500);
Serial.print("."); }
    Serial.println("\nIP Address: " + WiFi.localIP().toString());

    server.on("/", HTTP_GET, [] (AsyncWebServerRequest *request){

```

```

        request->send_P(200, "text/html", index_html);
    });

    ws.onEvent(onEvent);
    server.addHandler(&ws);
    server.begin();
}

void loop() {
    ws.cleanupClients();

    // 1. Eksekusi perangkat keras LED ON/OFF
    digitalWrite(ledPin, ledState ? HIGH : LOW);

    // 2. Baca Tombol Fisik (Debounce)
    int reading = digitalRead(buttonPin);
    if (reading != lastButtonState) {
        lastDebounceTime = millis();
    }
    if ((millis() - lastDebounceTime) > debounceDelay) {
        if (reading != buttonState) {
            buttonState = reading;
            if (buttonState == HIGH) {
                ledState = !ledState;
                notifyClients();
            }
        }
    }
    lastButtonState = reading;

    // 3. Baca DHT setiap 3 detik
    if ((millis() - lastTime) > 3000) {
        float t = dht.readTemperature();
        if(!isnan(t)) {
            currentTemp = String(t);
            notifyClients();
        }
        lastTime = millis();
    }
}
}

```

```

<input type="range" min="0" max="1023" value="0" id="pwmSlider" oninput="sendPWM(this.value)">

```

```

void handleWebSocketMessage(void *arg, uint8_t *data, size_t len) {
    AwsFrameInfo *info = (AwsFrameInfo*)arg;
    if (info->final && info->index == 0 && info->len == len && info->opcode == WS_TEXT) {
        data[len] = 0;
        String pesan = (char*)data;

        if (pesan == "toggle") {
            ledState = !ledState;
            notifyClients();
        }
    }
}

```

BAB 3. LANGKAH KERJA

1. Merangkai LED dan resistor 220 Ohm ke pin D1 (GPIO 5) sesuai Bab 1.
2. Mengisi nama dan kata sandi Wi-Fi pada variabel ssid dan password, lalu meng-upload program ke NodeMCU.
3. Membuka Serial Monitor (baud rate 115200) dan mencatat alamat IP yang ditampilkan NodeMCU.
4. Menghubungkan smartphone ke jaringan Wi-Fi yang sama dengan NodeMCU, lalu membuka alamat IP tersebut pada browser.
5. Menggeser slider dari nilai 0 sampai 1023 secara perlahan dan mengamati tingkat kecerahan LED.
6. Mengamati Serial Monitor untuk memastikan paket berformat pwm,nilai diterima dan diparsing dengan benar.
7. Menguji nilai batas: slider pada 0 (LED padam), pada 512 (LED setengah terang), dan pada 1023 (LED paling terang).

BAB 4. HASIL DAN PEMBAHASAN

4.1 Koneksi dan Tampilan Antarmuka

NodeMCU berhasil terhubung ke jaringan Wi-Fi dan menampilkan alamat IP pada Serial Monitor. Halaman web dengan slider berhasil dibuka dari smartphone dan status koneksi WebSocket berubah menjadi Terhubung. (ISI: tulis alamat IP yang muncul)

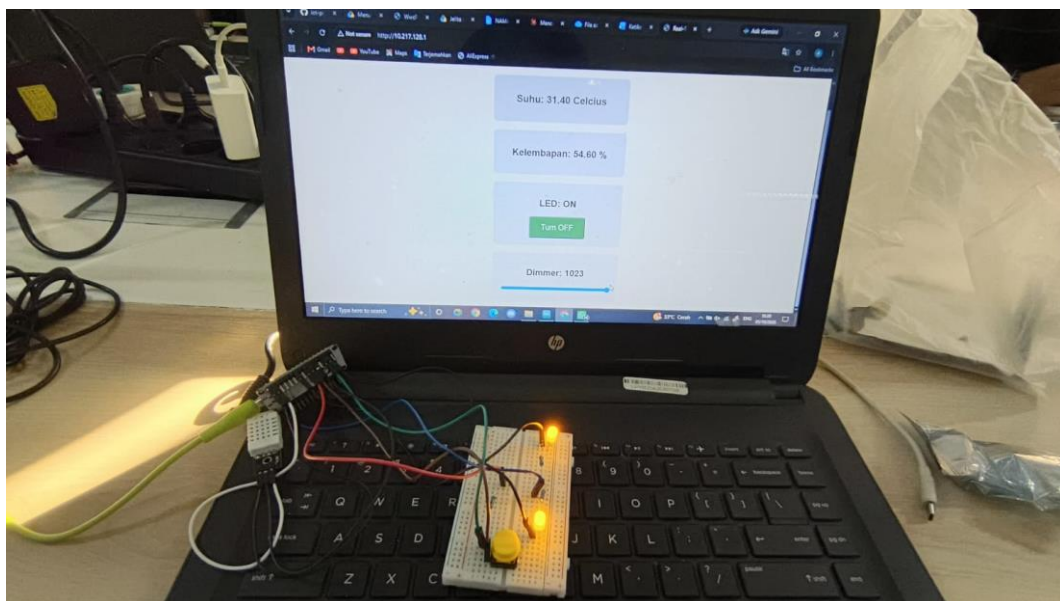
4.2 Pengiriman Paket dan Parsing

Setiap kali slider digeser, fungsi sendPWM(value) mengirim string berformat "pwm,nilai" melalui WebSocket, misalnya "pwm,512". Pada sisi NodeMCU,

`handleWebSocketMessage()` memeriksa awalan "pwm," menggunakan `startsWith()`, mengambil angka setelah koma menggunakan `substring()` dan `toInt()`, lalu membatasi nilainya pada rentang 0 sampai 1023 dengan `constrain()`. Nilai tersebut diteruskan ke `analogWrite(ledPin, nilaiPWM)`. Serial Monitor menampilkan nilai PWM yang diterima secara berurutan mengikuti gerakan slider. (ISI: sesuaikan dengan log aktual)

4.3 Pengujian Intensitas LED

Intensitas LED berubah secara halus mengikuti posisi slider. Pada nilai 0 LED padam, pada nilai sekitar 512 LED menyala dengan kecerahan sedang, dan pada nilai 1023 LED menyala paling terang. Hal ini menunjukkan bahwa PWM mengatur daya rata-rata ke LED melalui perubahan duty cycle, bukan melalui perubahan tegangan secara langsung. (ISI: tulis nilai yang diuji dan hasil pengamatan)

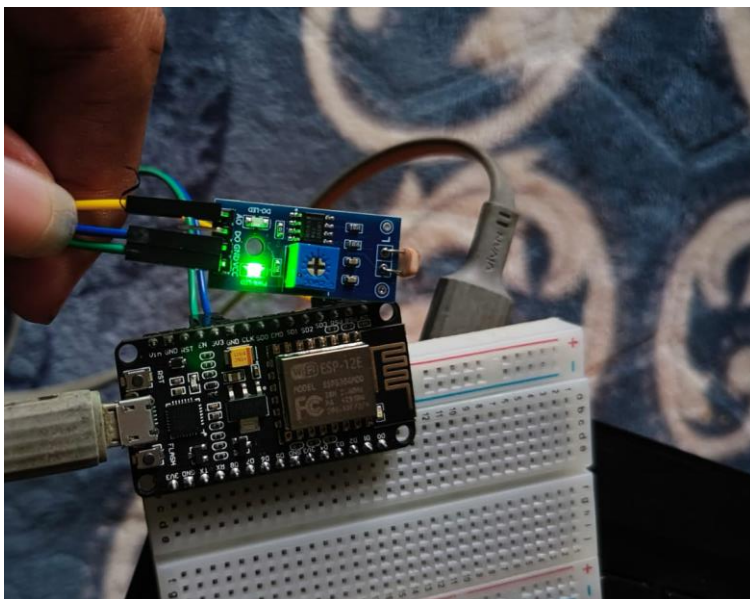
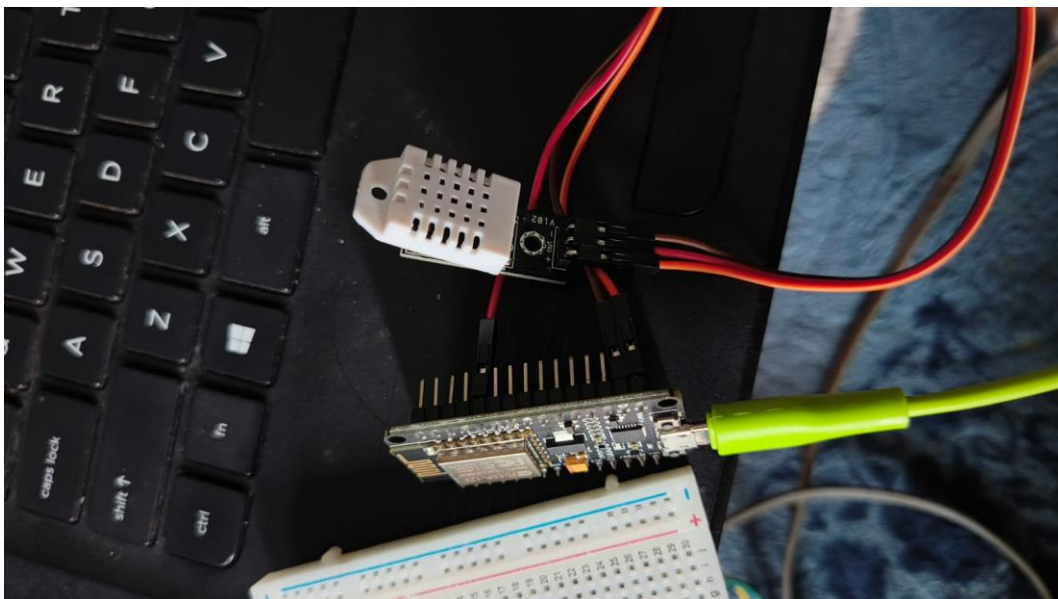


BAB 5. RANGKAIAN

Node 1 (Stasiun Lingkungan): pin DATA sensor DHT dihubungkan ke D4 (GPIO 2), VCC ke 3V3, GND ke GND.

Node 2 (Stasiun Cahaya): LDR dan resistor 10k Ohm dirangkai sebagai pembagi tegangan, titik tengahnya dihubungkan ke pin A0.

Node 3 (Hub Kontrol Aktuator): anoda LED ke D6 (GPIO 12), katoda LED ke salah satu kaki resistor 220 Ohm, kaki resistor lainnya ke GND.



BAB 6. PROGRAM

6.1 Program Node 1 (Sensor DHT)

// Node 1 – Stasiun Lingkungan (DHT11/DHT22 -> D4 / GPIO2)

```
#include <painlessMesh.h>
```

```
#include <DHT.h>
```

```
#define MESH_PREFIX “Redmi 12”
```

```
#define MESH_PASSWORD “22345678”
```

```
#define MESH_PORT 5555
```

```
#define DHTPIN 2 // D4 = GPIO2
```

```
#define DHTTYPE DHT22 // ganti DHT22 jika memakai DHT22
```

```
DHT dht(DHTPIN, DHTTYPE)
```

```
Scheduler userScheduler;
```

```
painlessMesh mesh;
```

```
void sendMessage();
```

```
Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER,  
&sendMessage);
```

```
Void sendMessage() {
```

```
Float t = dht.readTemperature();
```

```
Float h = dht.readHumidity();
```

```
If (isnan(t) || isnan(h)) {
```

```
    Serial.println("Gagal membaca DHT, paket dilewati");
```

```
    Return;
```

```
}
```

```
Char buf[96];
```

```
Snprintf(buf, sizeof(buf),
```

```
    "{\\"type\\":\\"suhu_node\\",\\"suhu\\":%.1f,\\"kelembapan\\":%.1f}", t, h);
```

```
String msg(buf);
```

```
Mesh.sendBroadcast(msg);
```

```
Serial.printf("[TX] %s\\n", msg.c_str());
```

```
}
```

```
Void receivedCallback(uint32_t from, String &msg) {
```

```
    Serial.printf("[RX] dari %u: %s\\n", from, msg.c_str());
```

```
}
```

```
Void newConnectionCallback(uint32_t nodeId) {
```

```
    Serial.printf("[MESH] Koneksi baru: %u\\n", nodeId);
```

```
}
```

```

Void changedConnectionCallback() {

    Serial.printf("[MESH] Topologi berubah: %s\n",
mesh.subConnectionJson().c_str());

}

```

```

Void nodeTimeAdjustedCallback(int32_t offset) {

    Serial.printf("[MESH] Waktu disesuaikan, offset=%d\n", offset);

}

```

```

Void setup() {

    Serial.begin(115200);

    Dht.begin();

    Mesh.setDebugMsgTypes(ERROR | STARTUP);

    Mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler,
MESH_PORT);

    Mesh.onReceive(&receivedCallback);

    Mesh.onNewConnection(&newConnectionCallback);

    Mesh.onChangedConnections(&changedConnectionCallback);

    Mesh.onNodeTimeAdjusted(&nodeTimeAdjustedCallback);

    userScheduler.addTask(taskSendMessage);
}

```

```

taskSendMessage.enable();

Serial.printf("Node 1 aktif, ID: %u\n", mesh.getNodeId());

}

Void loop() {

    Mesh.update();

}

```

6.2 Program Node 2 (Sensor LDR)

```

#include <painlessMesh.h>

#define MESH_PREFIX    "Redmi 12"
#define MESH_PASSWORD  "22345678"
#define MESH_PORT      5555

painlessMesh mesh;
Scheduler userScheduler;

void sendMessage() {
    int adcValue = analogRead(A0);
    String msg = "{\"tipe\":\"cahaya_node\",\"adc\":" +
String(adcValue) + "}";
    mesh.sendBroadcast(msg);
    Serial.println("Kirim: " + msg);
}

Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);

void setup() {
    Serial.begin(115200);
    mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);

    userScheduler.addTask(taskSendMessage);
    taskSendMessage.enable();
}

void loop() {
    mesh.update();
}

```

6.3 Program Node 3 (Hub Aktuator Cerdas)

```
#include <painlessMesh.h>
#include <ArduinoJson.h>

#define MESH_PREFIX    "Redmi 12"
#define MESH_PASSWORD  "22345678"
#define MESH_PORT      5555

#define LED_PIN 12

const float SUHU_BATAS = 35.0;
const int   ADC_GELAP  = 300;

Scheduler userScheduler;
painlessMesh mesh;

float suhuTerakhir = 0.0;
int   adcTerakhir  = 1023;

void evaluasiRule() {
    bool panas = suhuTerakhir > SUHU_BATAS;
    bool gelap = adcTerakhir < ADC_GELAP;
    bool ledOn  = panas || gelap;

    digitalWrite(LED_PIN, ledOn ? HIGH : LOW);
    Serial.printf("[RULE] suhu=%.1f (batas %.1f) adc=%d (batas %d) ->
panas=%d gelap=%d -> LED %s\n",
                  suhuTerakhir, SUHU_BATAS, adcTerakhir, ADC_GELAP,
                  panas, gelap, ledOn ? "ON" : "OFF");
}

void receivedCallback(uint32_t from, String &msg) {
    Serial.printf("[RX] dari %u: %s\n", from, msg.c_str());

    StaticJsonDocument<200> doc;
    DeserializationError err = deserializeJson(doc, msg);
    if (err) {
        Serial.printf("[JSON] Gagal parsing: %s\n", err.c_str());
        return;
    }

    const char* tipe = doc["tipe"] | "";
```

```

    if (strcmp(tipe, "suhu_node") == 0) {
        suhuTerakhir = doc["suhu"] | suhuTerakhir;
        Serial.printf("[DATA] Node 1: suhu=%.1f\n", suhuTerakhir);
    } else if (strcmp(tipe, "cahaya_node") == 0) {
        adcTerakhir = doc["adc"] | adcTerakhir;
        Serial.printf("[DATA] Node 2: adc=%d\n", adcTerakhir);
    } else {
        Serial.println("[JSON] Tipe tidak dikenal");
        return;
    }

    evaluasiRule();
}

void newConnectionCallback(uint32_t nodeId) {
    Serial.printf("[MESH] Koneksi baru: %u\n", nodeId);
}

void changedConnectionCallback() {
    Serial.printf("[MESH] Topologi: %s\n",
mesh.subConnectionJson().c_str());
    auto nodes = mesh.getNodeList(false);
    Serial.printf("[MESH] Jumlah node terhubung: %u\n",
(unsigned)nodes.size());
}

void nodeTimeAdjustedCallback(int32_t offset) {}

void setup() {
    Serial.begin(115200);
    pinMode(LED_PIN, OUTPUT);

    Serial.println("Tes LED...");
    digitalWrite(LED_PIN, HIGH);
    delay(2000);
    digitalWrite(LED_PIN, LOW);
    Serial.println("Tes LED selesai");

    mesh.setDebugMsgTypes(ERROR | STARTUP);
    mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);
    mesh.onReceive(&receivedCallback);
    mesh.onNewConnection(&newConnectionCallback);
    mesh.onChangedConnections(&changedConnectionCallback);
}

```

```

mesh.onNodeTimeAdjusted(&nodeTimeAdjustedCallback);

Serial.printf("Node 3 aktif, ID: %u\n", mesh.getNodeId());
}

void loop() {
    mesh.update();
}

```

BAB 7. LANGKAH KERJA

1. Merangkai ketiga node sesuai skema pada Bab 5.
2. Meng-upload program Node 1, Node 2, dan Node 3 ke masing-masing NodeMCU.
3. Menghidupkan ketiga node berdekatan dan membuka Serial Monitor (baud rate 115200) untuk memastikan jaringan mesh terbentuk.
4. Mengamati data JSON dari Node 1 dan Node 2 yang diterima dan di-parsing oleh Node 3.
5. Menguji rule engine: memanaskan sensor DHT (misalnya dengan hembusan napas atau udara hangat) dan menutup LDR dengan tangan, lalu mengamati status LED.
6. Eksperimen multi-hop: menjauhkan Node 1 dan Node 3 sampai di luar jangkauan langsung, menempatkan Node 2 di tengah, lalu mengamati Serial Monitor Node 3.
7. Eksperimen self-healing: mematikan daya Node 2 dan mengamati log `changedConnectionCallback()` pada Serial Monitor Node 3.

BAB 8. HASIL DAN PEMBAHASAN

8.1 Pembentukan Jaringan Mesh

Ketiga node berhasil membentuk jaringan mesh tanpa router Wi-Fi eksternal. Pada Serial Monitor tiap node muncul log koneksi baru (`newConnectionCallback`) yang menunjukkan node saling menemukan secara otomatis menggunakan `MESH_PREFIX`, `MESH_PASSWORD`, dan `MESH_PORT` yang sama.

8.2 Pengiriman dan Parsing Data JSON

Node 1 mengirim data suhu dan kelembapan setiap 3 detik dalam format {"tipe":"suhu_node","suhu":(ISI),"kelembapan":(ISI)}. Node 2 mengirim nilai ADC LDR setiap 3 detik dalam format "{\"tipe\":\"cahaya_node\",\"adc\":\" + String(adcValue) + \"}\"; .Node 3 menerima paket tersebut melalui receivedCallback(), membedahnya dengan ArduinoJson, lalu membedakan sumber data berdasarkan field "tipe". Tidak ada pesan "Gagal parsing JSON", sehingga parsing berjalan benar.

```
12:00:04.757 -> [TX] {"tipe":"suhu_node","suhu":34.8,"kelembapan":71.0}
12:00:06.062 -> [RX] dari 571611746: {"tipe":"cahaya_node","adc":781}
12:00:07.791 -> [TX] {"tipe":"suhu_node","suhu":34.8,"kelembapan":69.5}
12:00:09.077 -> [RX] dari 571611746: {"tipe":"cahaya_node","adc":785}
12:00:10.757 -> [TX] {"tipe":"suhu_node","suhu":34.8,"kelembapan":69.0}
12:00:12.081 -> [RX] dari 571611746: {"tipe":"cahaya_node","adc":784}
12:00:13.781 -> [TX] {"tipe":"suhu_node","suhu":34.8,"kelembapan":67.7}
12:00:15.077 -> [RX] dari 571611746: {"tipe":"cahaya_node","adc":784}
12:00:16.794 -> [TX] {"tipe":"suhu_node","suhu":34.8,"kelembapan":67.4}
12:00:18.046 -> [RX] dari 571611746: {"tipe":"cahaya_node","adc":785}
12:00:19.799 -> [TX] {"tipe":"suhu_node","suhu":34.8,"kelembapan":67.3}
12:00:21.086 -> [RX] dari 571611746: {"tipe":"cahaya_node","adc":785}
12:00:22.759 -> [TX] {"tipe":"suhu_node","suhu":34.8,"kelembapan":67.2}
12:00:24.088 -> [RX] dari 571611746: {"tipe":"cahaya_node","adc":784}
12:00:25.757 -> [TX] {"tipe":"suhu_node","suhu":34.8,"kelembapan":67.2}
12:00:27.045 -> [RX] dari 571611746: {"tipe":"cahaya_node","adc":788}
12:00:28.798 -> [TX] {"tipe":"suhu_node","suhu":34.8,"kelembapan":67.1}
12:00:30.046 -> [RX] dari 571611746: {"tipe":"cahaya_node","adc":792}
12:00:31.781 -> [TX] {"tipe":"suhu_node","suhu":34.7,"kelembapan":66.8}
12:00:33.043 -> [RX] dari 571611746: {"tipe":"cahaya_node","adc":782}
12:00:34.756 -> [TX] {"tipe":"suhu_node","suhu":34.7,"kelembapan":66.9}
12:00:36.066 -> [RX] dari 571611746: {"tipe":"cahaya_node","adc":775}
12:00:37.778 -> [TX] {"tipe":"suhu_node","suhu":34.7,"kelembapan":67.1}
12:00:39.046 -> [RX] dari 571611746: {"tipe":"cahaya_node","adc":776}
12:00:40.757 -> [TX] {"tipe":"suhu_node","suhu":34.7,"kelembapan":67.1}
12:00:42.080 -> [RX] dari 571611746: {"tipe":"cahaya_node","adc":776}
12:00:43.786 -> [TX] {"tipe":"suhu_node","suhu":34.7,"kelembapan":67.0}
```

```
Kirim: {"tipe": "cahaya_node", "adc": 164}
Kirim: {"tipe": "cahaya_node", "adc": 244}
Kirim: {"tipe": "cahaya_node", "adc": 965}
Kirim: {"tipe": "cahaya_node", "adc": 319}
Kirim: {"tipe": "cahaya_node", "adc": 942}
Kirim: {"tipe": "cahaya_node", "adc": 992}
Kirim: {"tipe": "cahaya_node", "adc": 972}
Kirim: {"tipe": "cahaya_node", "adc": 937}
Kirim: {"tipe": "cahaya_node", "adc": 937}
Kirim: {"tipe": "cahaya_node", "adc": 964}
Kirim: {"tipe": "cahaya_node", "adc": 945}
Kirim: {"tipe": "cahaya_node", "adc": 942}
Kirim: {"tipe": "cahaya_node", "adc": 951}
Kirim: {"tipe": "cahaya_node", "adc": 944}
Kirim: {"tipe": "cahaya_node", "adc": 976}
Kirim: {"tipe": "cahaya_node", "adc": 972}
Kirim: {"tipe": "cahaya_node", "adc": 997}
Kirim: {"tipe": "cahaya_node", "adc": 1001}
Kirim: {"tipe": "cahaya_node", "adc": 1010}
Kirim: {"tipe": "cahaya_node", "adc": 1010}
Kirim: {"tipe": "cahaya_node", "adc": 1004}
```

Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

```
[DATA] Node 2: adc=643
[RULE] suhu=34.0 (batas 35.0) adc=643 (batas 300) -> panas=0 gelap=0 -> LED OFF
[RX] dari 571611746: {"tipe": "cahaya_node", "adc": 657}
[DATA] Node 2: adc=657
[RULE] suhu=34.0 (batas 35.0) adc=657 (batas 300) -> panas=0 gelap=0 -> LED OFF
[RX] dari 571611746: {"tipe": "cahaya_node", "adc": 674}
[DATA] Node 2: adc=674
[RULE] suhu=34.0 (batas 35.0) adc=674 (batas 300) -> panas=0 gelap=0 -> LED OFF
[RX] dari 571611746: {"tipe": "cahaya_node", "adc": 282}
[DATA] Node 2: adc=282
[RULE] suhu=34.0 (batas 35.0) adc=282 (batas 300) -> panas=0 gelap=1 -> LED ON
[RX] dari 571611746: {"tipe": "cahaya_node", "adc": 282}
[DATA] Node 2: adc=282
[RULE] suhu=34.0 (batas 35.0) adc=282 (batas 300) -> panas=0 gelap=1 -> LED ON
[RX] dari 571611746: {"tipe": "cahaya_node", "adc": 290}
[DATA] Node 2: adc=290
[RULE] suhu=34.0 (batas 35.0) adc=290 (batas 300) -> panas=0 gelap=1 -> LED ON
[RX] dari 571611746: {"tipe": "cahaya_node", "adc": 269}
[DATA] Node 2: adc=269
[RULE] suhu=34.0 (batas 35.0) adc=269 (batas 300) -> panas=0 gelap=1 -> LED ON
[RX] dari 571611746: {"tipe": "cahaya_node", "adc": 290}
[DATA] Node 2: adc=290
[RULE] suhu=34.0 (batas 35.0) adc=290 (batas 300) -> panas=0 gelap=1 -> LED ON
```